



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Tarea 5

ALUMNOS:

Alejandro Espino Gutiérrez

PROFESORES:

Javier León Cotonieto
Jose Manuel Evangelista Tiburcio
Adrián Martínez Manzo

ASIGNATURA

Sistemas Operativos

22 de octubre de 2026

1 Algoritmo y Metodología

Para esta tarea elegí el problema de Cadena de Referencia. Una cadena de referencia es una secuencia derivada de la traza de ejecución (la cual sólo tiene los accesos a direcciones de memoria) que convierte los accesos a direcciones a páginas de memoria.

El problema del libro es el siguiente

A continuación se detalla una traza de ejecución de un fragmento de programa para una computadora con páginas de 512 bytes. El programa está en la dirección 1020, y su apuntador de pila está en 8192 (la pila crece hacia 0). De la cadena de referencias de páginas generada por este programa. Cada instrucción ocupa 4 bytes (1 palabra), incluyendo constantes inmediatas. Tanto las instrucciones como referencias a datos cuentan en la cadena de referencias.

```
Load word 6144 into register 0
Push register 0 onto the stack
Call a procedure at 5120, stacking the return address
Subtract the immediate constant 16 from the stack pointer
Compare the actual parameter to the immediate constant 4
Jump if equal to 5152
```

Y la hoja de cálculo de tareas nos pide un algoritmo para derivar la cadena de referencias a partir de una traza de ejecución

Entonces primero debemos traducir este programa a una traza de ejecución, para lo cual escribimos los accesos que hace el programa para lectura o escritura

```
1020
6144
1024
8188
1028
8184
5120
5124
8188
5128
```

Entonces ahora veamos el algoritmo de forma general que vamos a usar para obtener la cadena de referencias.

- Crear un arreglo dinámico para guardar direcciones de memoria

- Iterar sobre el arreglo con las direcciones, dividiendo cada dirección sobre el tamaño de página para obtener el número de página
- Imprime el número de cada página

Una vez visto el algoritmo general, se implementó en `C`, creando un método `parsear_addrs` además del `main`. Vamos a usar un archivo de texto plano donde guardemos la traza en el formato que vimos anteriormente, entonces la función `parsear_addrs` va a leer ese archivo, crea el arreglo dinámico e inserta ahí cada dirección que lea. Una vez hecho esto, la función `main` va a realizar la división sobre el tamaño de páginas e imprimir el resultado.

2 Resultados y Análisis

Para compilar el código debemos colocarnos en el directorio raíz de la práctica y ejecutar el comando

```
gcc cadena.c -o cadena
```

Y para ejecutarlo debemos hacer:

```
./cadena <tamaño_pagina> <archivo_traza>
```

Entonces tenemos en un archivo `direcciones.txt` la traza del programa del ejercicio del libro, y al compilar y ejecutar (con el tamaño de página 512 y la dirección del archivo) obtenemos el siguiente resultado

```
alejandroespino@2806-107e-0019-5f8e-5d53-7a81-42d4-92c6:~/Escritorio/Escuela/ciencias-computacion/2027-1/sistemas-operativos/tareas/5$ gcc src/cadena.c -o cadena
alejandroespino@2806-107e-0019-5f8e-5d53-7a81-42d4-92c6:~/Escritorio/Escuela/ciencias-computacion/2027-1/sistemas-operativos/tareas/5$ ./cadena 512 direcciones.txt
Tamaño de pagina: 512 bytes
Secuencia de referencias de pagina: 1 12 2 15 2 15 10 10 15 10
```

Entonces la secuencia de referencias es:

```
1 12 2 15 2 15 10 10 15 10
```

Analizando el código, notemos que la complejidad de nuestro algoritmo es $O(n)$ para n direcciones en el archivo, también se consideraron algunos errores que puede cometer el usuario al usar el programa, como un mal formato de args o un mal formato del archivo, y podemos ver que en efecto obtenemos las páginas a las que pertenece cada dirección de memoria.

3 Conclusión

En conclusión, vemos que el algoritmo traduce de forma correcta las direcciones de memoria de la traza a páginas de memoria, lo cual es bastante útil en el desarrollo de Sistemas Operativos, pues esto en la práctica se integraría con un programa de gestión de memoria con paginación, específicamente un algoritmo que use algún tipo de paginación, el cual permite a programas usuarios del SO acceder a una API donde no tengan que manejar la memoria desde la aplicación, sino esto estaría automatizado por parte del SO. El manejo de memoria en conjunto le brinda a los procesos protección en cuanto a lectura o escritura de su memoria por parte de otros procesos sin autorización, así como manejo de posibles fallos en la memoria, para que las aplicaciones puedan ejecutarse de forma modular y segura en cuanto a memoria.

4 Referencias

- Tannenbaum, A. S., Bos, H. (s. f.). Modern Operating Systems (4.a ed.).

5 Repositorio

El código está en `https://github.com/alejandro-espino-gut/tarea5-SO.git`